

Hàng đợi thông điệp

Message Queues

AGENDA

01

Mô hình Producer-Consumer & Publish-Subscribe

02

Backpressure — xử lý khi hệ thống bị quá tải

03

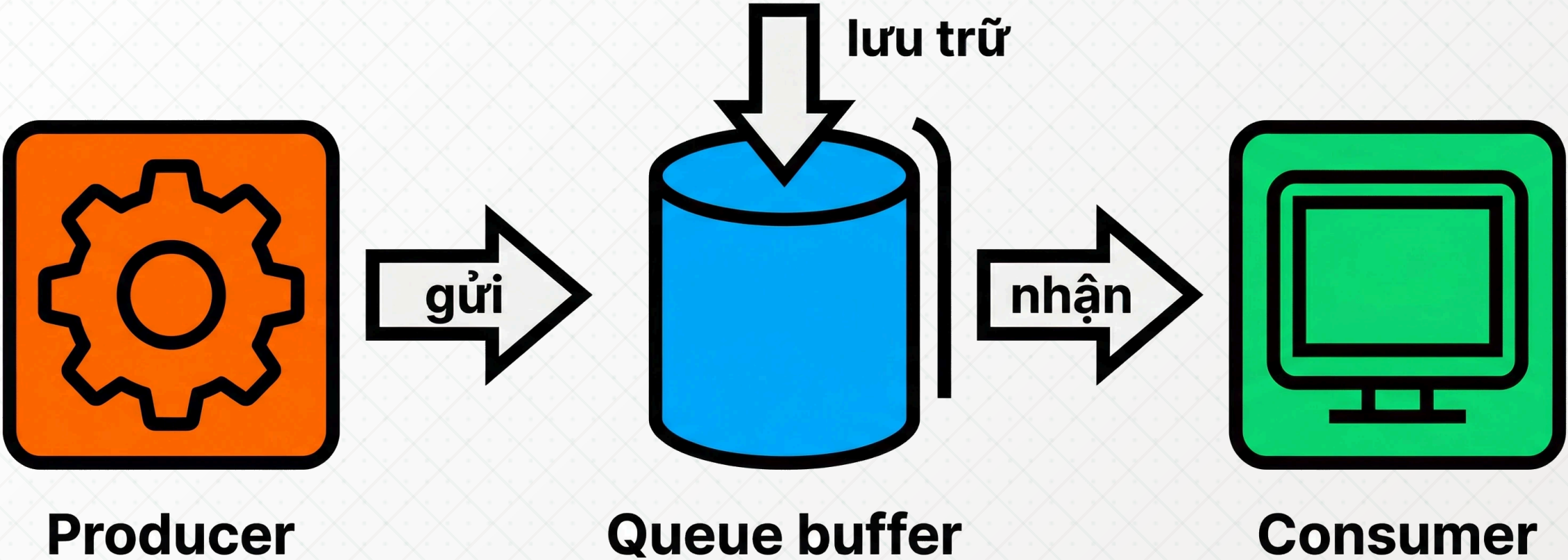
Các hệ thống thực tế: Kafka & RabbitMQ

Mô hình Producer-Consumer & Publish-Subscribe

Message queue là gì?

Message queue là một bộ đệm trung gian lưu trữ thông điệp giữa các thành phần hệ thống, cho phép giao tiếp **bất đồng bộ** và tách rời.

- ☕ **Producer** gửi thông điệp vào hàng đợi mà không cần chờ phản hồi
- ☕ **Consumer** lấy và xử lý thông điệp theo tốc độ của mình
- ☕ Đảm bảo **độ bền** (durability): thông điệp không bị mất khi consumer ngừng hoạt động
- ☕ Cho phép hệ thống **mở rộng theo chiều ngang** độc lập ở từng tầng

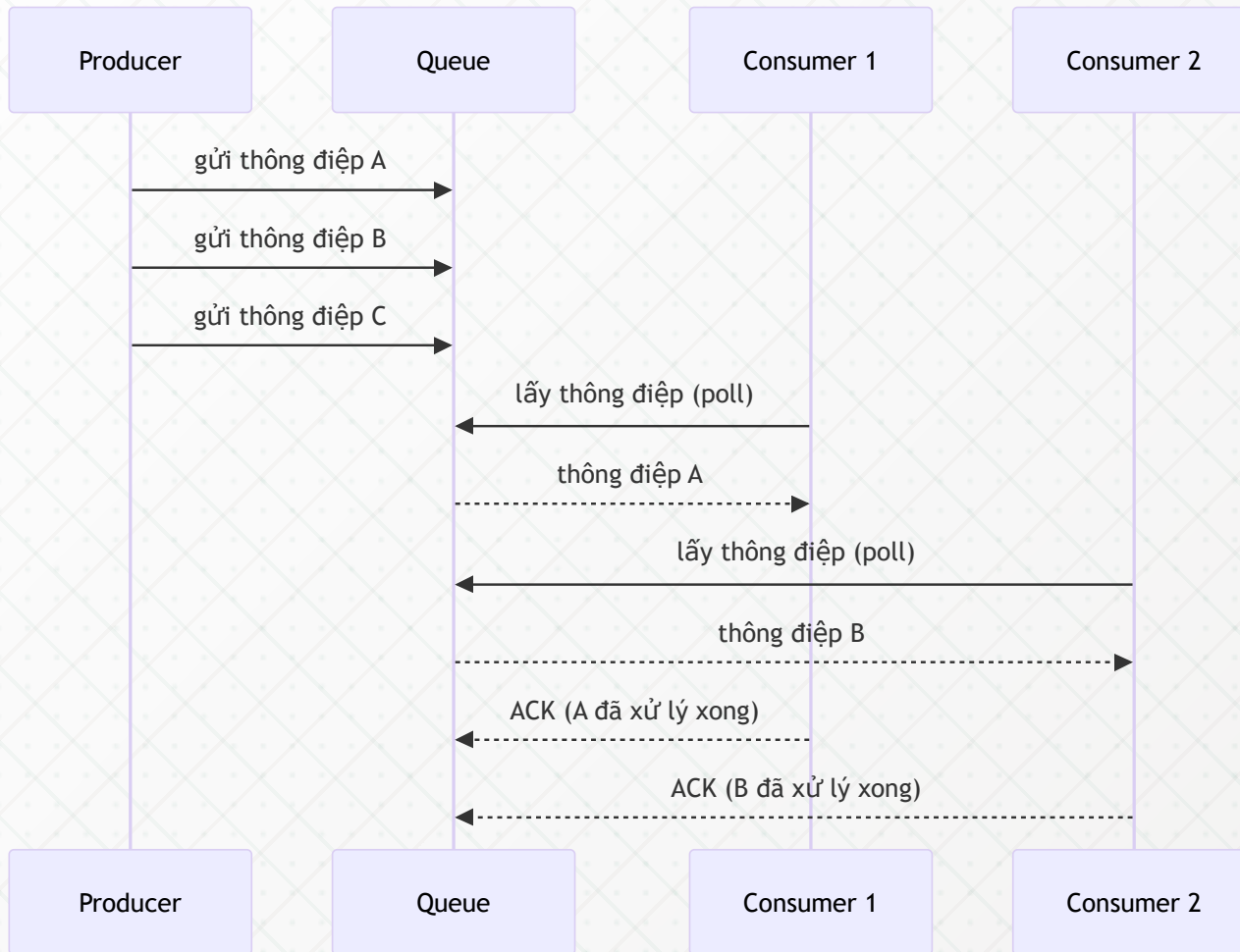


Mô hình Producer-Consumer

Một hoặc nhiều **producer** tạo thông điệp; một hoặc nhiều **consumer** tranh giành nhau để lấy và xử lý — mỗi thông điệp chỉ được xử lý **đúng một lần**.

- ☕ Producer và consumer hoạt động **độc lập** về tốc độ và vòng đời
- ☕ Queue hoạt động theo cơ chế **FIFO** (First In, First Out)
- ☕ Nhiều consumer có thể cùng kéo từ một queue để **tăng throughput**
- ☕ Thông điệp bị **xóa khỏi queue** sau khi consumer xác nhận đã xử lý (*acknowledgement*)
- ☕ Phù hợp với tác vụ nặng cần phân phối công việc: xử lý ảnh, gửi email, thanh toán

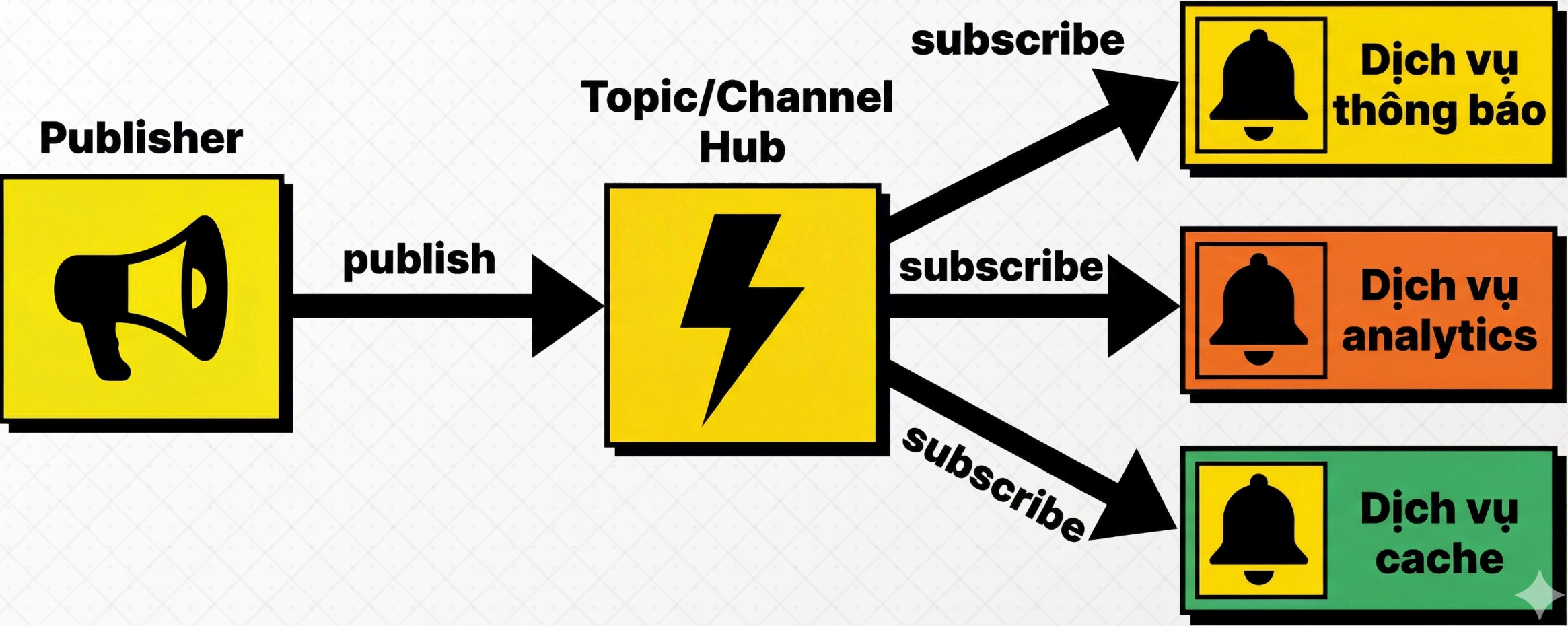
Sơ đồ: luồng Producer → Queue → Consumer



Mô hình Publish-Subscribe

Publisher phát thông điệp lên một *topic/channel*; tất cả **subscriber** đang lắng nghe đều nhận được *bản sao* của thông điệp đó.

- ☕ Một thông điệp có thể đến **nhều subscriber** cùng lúc (fan-out)
- ☕ Publisher **không biết** subscriber là ai — tách rời hoàn toàn
- ☕ Subscriber đăng ký nhận theo *topic* hoặc *pattern* phù hợp với nhu cầu
- ☕ Phù hợp với: thông báo sự kiện, cập nhật real-time, đồng bộ dữ liệu đa dịch vụ



So sánh Producer-Consumer và Publish-Subscribe

Hai mô hình giải quyết hai bài toán khác nhau — chọn đúng mô hình quyết định thiết kế hệ thống.

Producer-Consumer

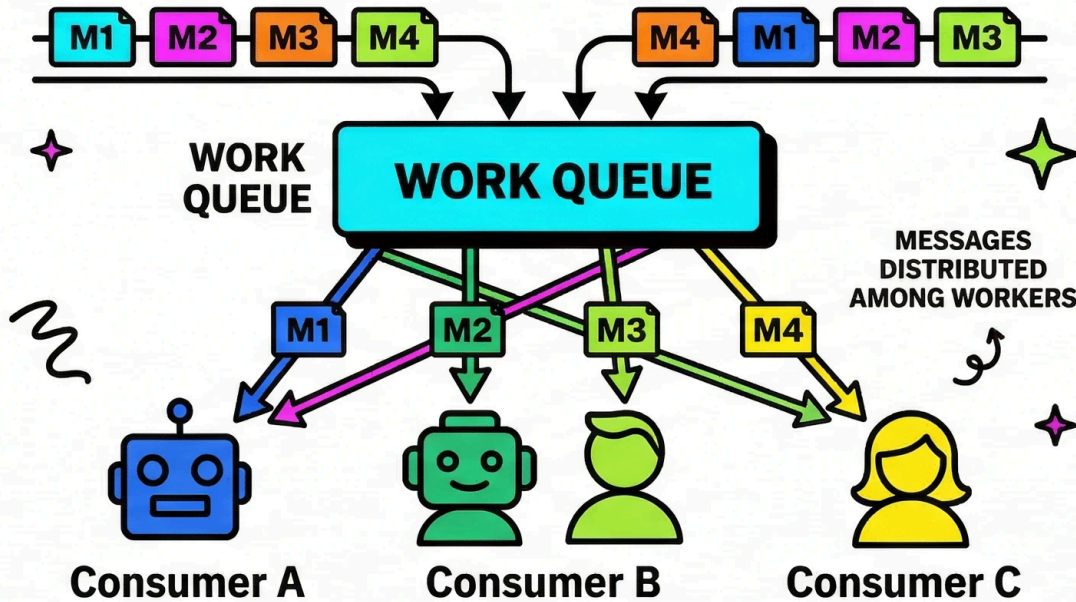
- Mỗi thông điệp xử lý **đúng một lần**
- Consumer **cạnh tranh** nhau lấy thông điệp
- Dùng để **phân phối tác vụ** (task distribution)
- Ví dụ: hàng đợi xử lý đơn hàng, encode video

Publish-Subscribe

- Mỗi thông điệp đến **tất cả** subscriber
- Subscriber nhận **bản sao độc lập**
- Dùng để **phát sóng sự kiện** (event broadcasting)
- Ví dụ: thông báo real-time, đồng bộ cache

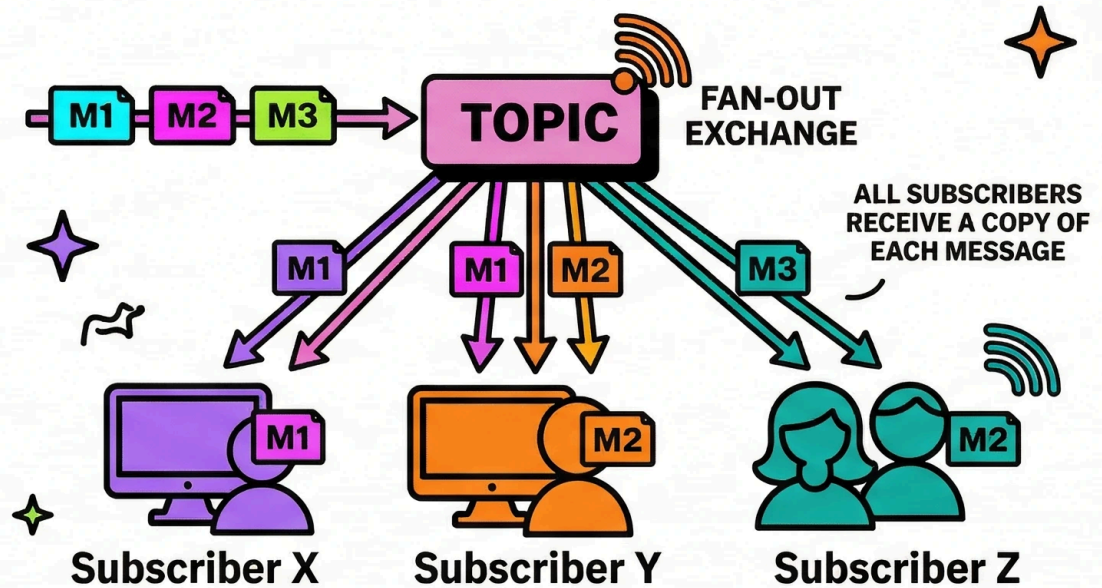
MESSAGE DELIVERY MODELS

SINGLE QUEUE WITH
COMPETING CONSUMERS



1 thông điệp → 1 consumer

TOPIC BROADCASTING TO
MULTIPLE SUBSCRIBERS



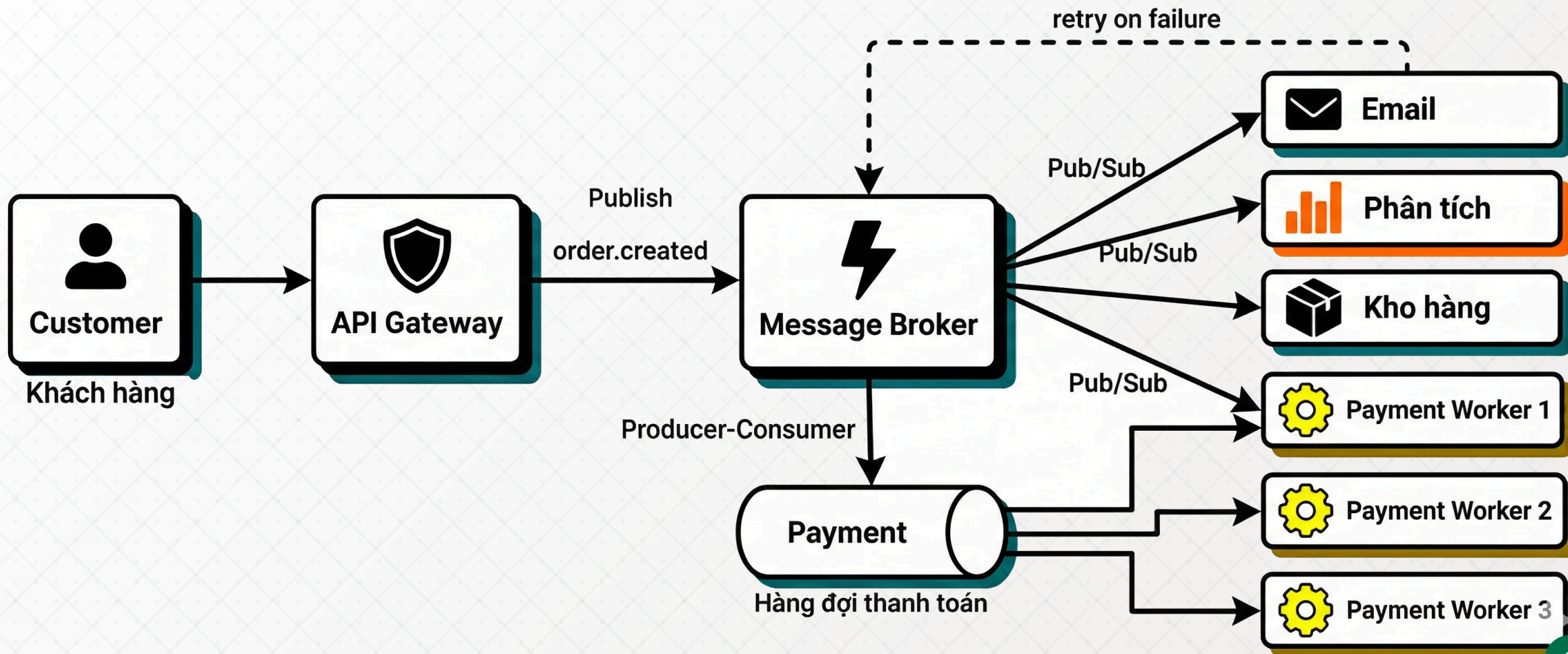
1 thông điệp → N subscriber

Hệ thống đặt hàng e-commerce

Kết hợp cả hai mô hình giúp hệ thống đặt hàng *mở rộng tốt* và *chịu lỗi tốt*.

- ☕ Khách đặt hàng → **API** đẩy sự kiện `order.created` vào message broker
- ☕ **Pub/Sub**: dịch vụ email, analytics, kho hàng đều nhận được sự kiện
- ☕ **Producer-Consumer**: dịch vụ thanh toán có queue riêng, nhiều worker xử lý song song
- ☕ Nếu dịch vụ email sập → thông điệp vẫn chờ trong queue, không mất dữ liệu
- ☕ Ngày sale lớn: chỉ cần **scale thêm worker** mà không đụng đến API hay frontend

Sơ đồ: hệ thống đặt hàng e-commerce



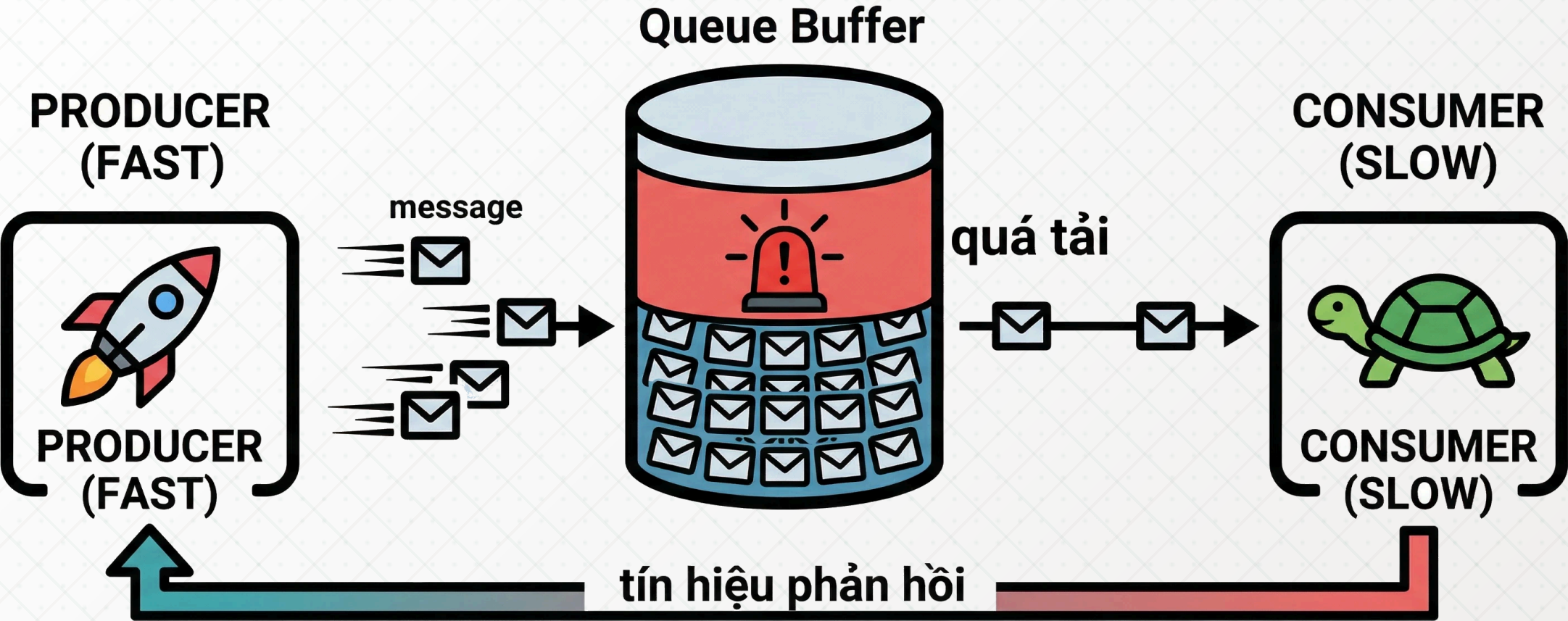
Backpressure

Xử lý khi hệ thống bị quá tải

Backpressure là gì?

Backpressure là cơ chế *phản hồi ngược* từ consumer về producer khi consumer không thể xử lý thông điệp đủ nhanh, buộc producer phải giảm tốc độ gửi.

- ☕ Xảy ra khi **tốc độ producer > tốc độ consumer** kéo dài
- ☕ Không có backpressure → queue phình to → **hết bộ nhớ** hoặc **latency tăng vọt**
- ☕ Backpressure là tín hiệu: "**Tôi đang quá tải – hãy chậm lại**"
- ☕ Khác với **rate limiting** (giới hạn từ phía trên); backpressure là áp lực từ phía dưới



Các chiến lược xử lý backpressure

Không có chiến lược nào tốt nhất cho mọi tình huống — lựa chọn phụ thuộc vào *yêu cầu nghiệp vụ*.

- ☕ **Blocking (chặn)**: producer bị dừng lại cho đến khi có chỗ trong queue — đảm bảo không mất dữ liệu
- ☕ **Dropping (bỏ)**: loại bỏ thông điệp mới khi queue đầy — chấp nhận mất dữ liệu để bảo vệ hệ thống
- ☕ **Sampling (lấy mẫu)**: chỉ giữ lại một phần thông điệp (ví dụ: 1 trong 10) — dùng cho metrics, logs
- ☕ **Buffering có giới hạn**: cho phép queue tăng đến ngưỡng tối đa, sau đó áp dụng blocking hoặc dropping
- ☕ **Scale consumer**: tự động thêm consumer khi queue vượt ngưỡng (*auto-scaling*)

Hậu quả khi bỏ qua backpressure

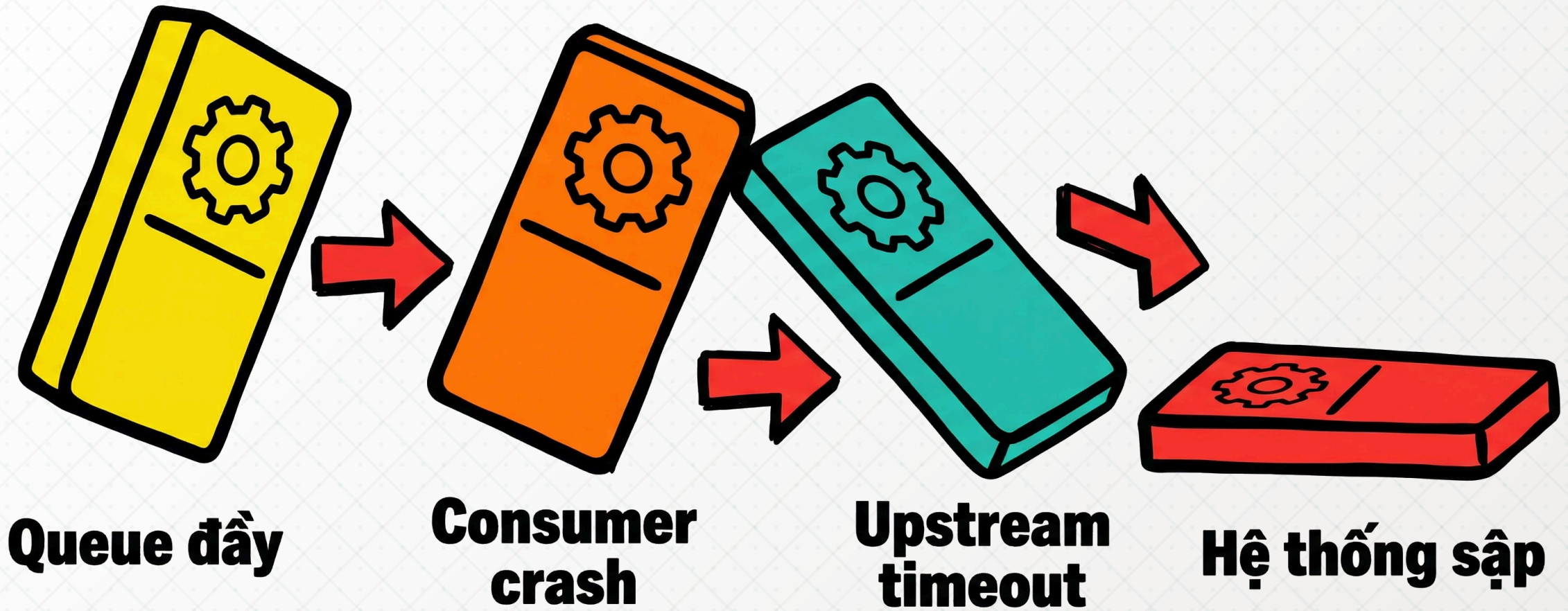
Bỏ qua backpressure dẫn đến ***cascade failure*** — sự cố lan rộng không kiểm soát được.

Chuỗi sự kiện thường gặp:

- ☕ Queue tăng không giới hạn → **OOM** (Out of Memory)
- ☕ Latency tăng → ***timeout*** lan sang các dịch vụ upstream
- ☕ Consumer bị quá tải → crash → queue tích lũy thêm
- ☕ Hệ thống sập theo ***hiệu ứng domino***

Dấu hiệu cần theo dõi (metrics):

- ☕ ***Queue depth*** tăng liên tục
- ☕ ***Consumer lag*** ngày càng lớn
- ☕ ***Memory usage*** của broker tăng bất thường



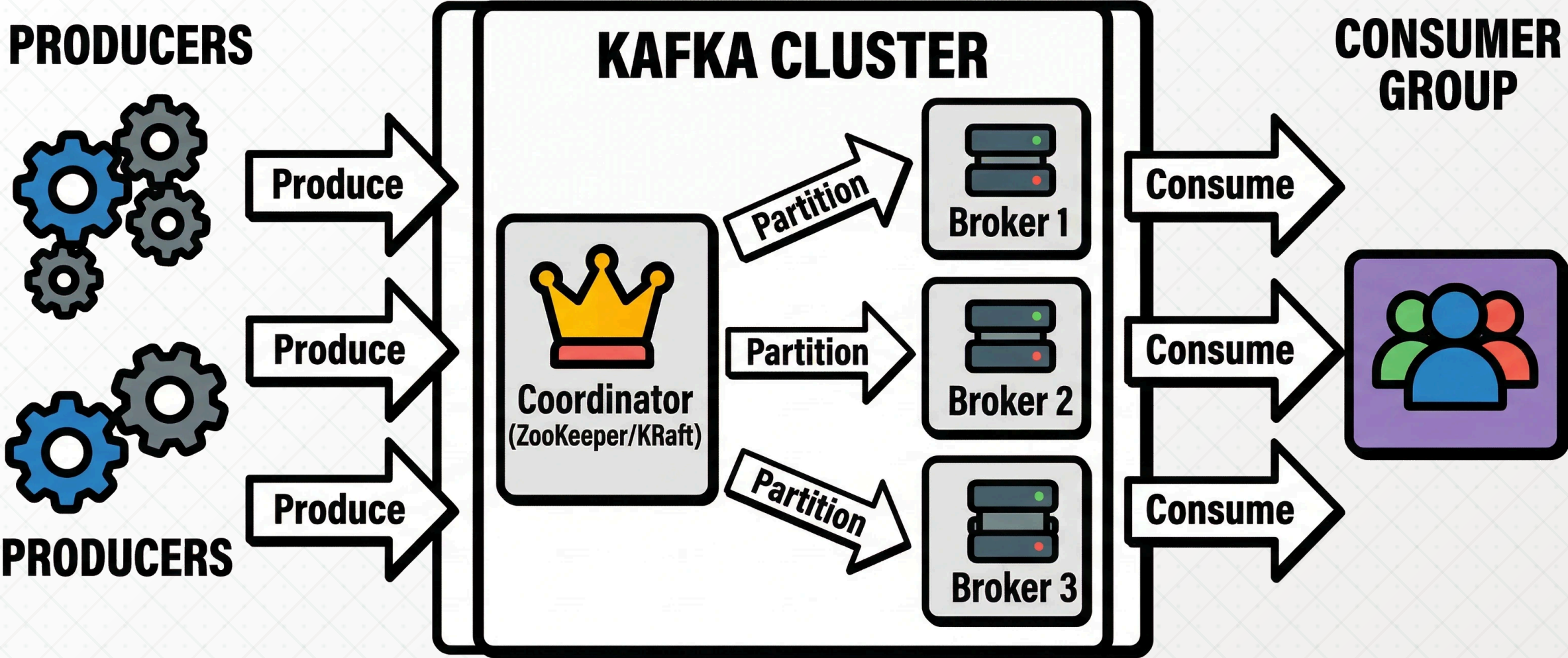
Các hệ thống thực tế

Kafka & RabbitMQ

Apache Kafka — giới thiệu

Kafka là ***distributed event streaming platform*** được thiết kế cho ***throughput cực cao*** và lưu trữ sự kiện bền vững theo thời gian.

- ☕ Phát triển bởi LinkedIn năm 2011, mã nguồn mở qua Apache Foundation
- ☕ Xử lý **hàng triệu thông điệp mỗi giây** với độ trễ thấp
- ☕ Thông điệp được lưu trên **đĩa** và giữ trong thời gian cấu hình (mặc định 7 ngày)
- ☕ Consumer **không xóa** thông điệp sau khi đọc — có thể đọc lại (***replay***)
- ☕ Ứng dụng: event sourcing, data pipeline, real-time analytics, log aggregation



Kafka: Topic, Partition và Consumer Group

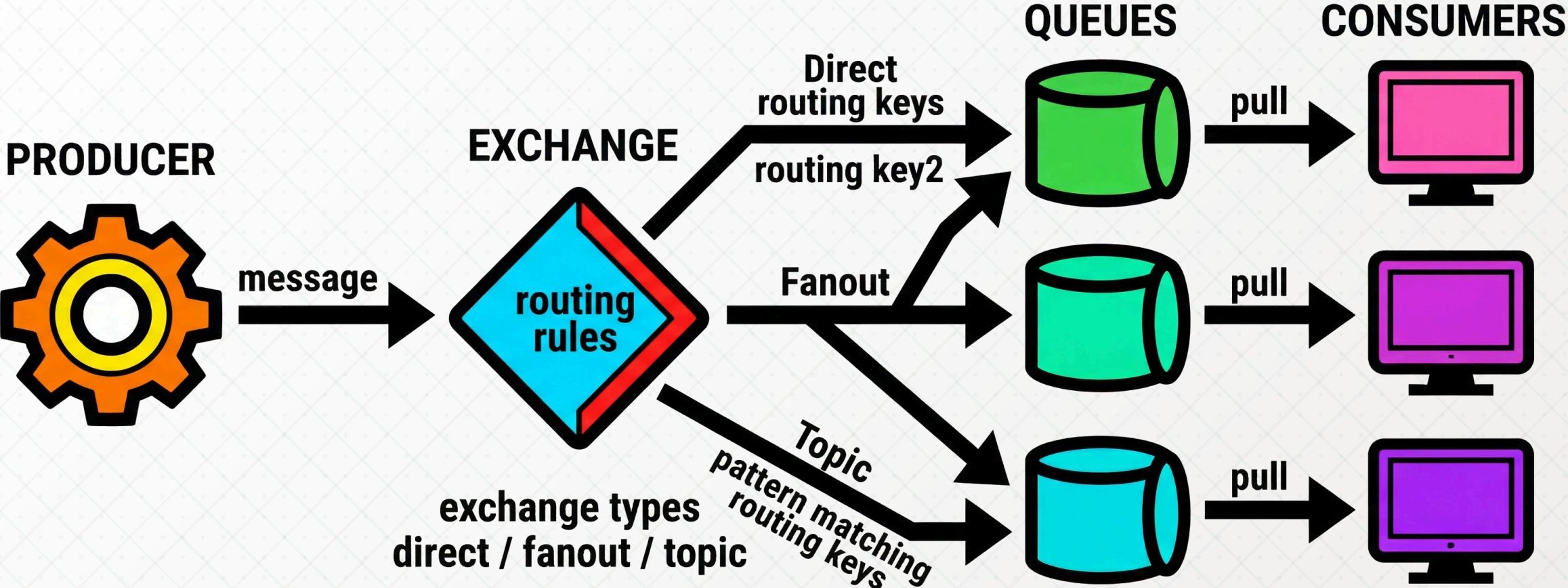
Topic là kênh dữ liệu; **Partition** là đơn vị song song hóa; **Consumer Group** là cơ chế phân phối tải giữa các consumer.

- ☕ **Topic**: luồng dữ liệu có tên, ví dụ `order.created`, `user.login`
- ☕ **Partition**: topic được chia thành nhiều partition để xử lý song song — mỗi partition là một log tuần tự
- ☕ **Offset**: vị trí của thông điệp trong partition; consumer tự quản lý offset của mình
- ☕ **Consumer Group**: nhiều consumer cùng group chia nhau các partition — *scale ngang tự nhiên*
- ☕ Số consumer tối đa hiệu quả = số partition của topic

RabbitMQ – giới thiệu

RabbitMQ là **message broker** truyền thống, linh hoạt về **routing**, tối ưu cho các luồng thông điệp **phức tạp** và **đòi hỏi độ tin cậy cao**.

- ☕ Phát triển bởi Pivotal (nay là VMware), dựa trên giao thức **AMQP**
- ☕ Hỗ trợ nhiều kiểu routing: **direct**, **fanout**, **topic**, **headers**
- ☕ Thông điệp **bị xóa** sau khi consumer ACK — không có cơ chế replay mặc định
- ☕ Hỗ trợ **priority queue**, **dead letter queue**, **message TTL** linh hoạt
- ☕ Phù hợp với: task queue, RPC qua message, workflow phức tạp nhiều bước



RabbitMQ: Exchange, Queue và Binding

Exchange nhận thông điệp từ producer và quyết định **routing** đến queue nào dựa trên **binding key**.

- ☕ **Exchange** là điểm nhận trung tâm — không lưu thông điệp, chỉ điều hướng
- ☕ **Binding**: quy tắc liên kết exchange với queue theo **routing key** hoặc **pattern**
- ☕ **Direct exchange**: routing key khớp chính xác → đúng một queue
- ☕ **Fanout exchange**: phát đến **tất cả** queue đang bind — không cần routing key
- ☕ **Topic exchange**: routing key theo **pattern** (`*.error`, `payment.#`) — linh hoạt nhất

So sánh Kafka và RabbitMQ

Kafka và RabbitMQ không phải đối thủ — chúng giải quyết *hai lớp bài toán khác nhau*.

Apache Kafka

- Mô hình: *distributed log* (log phân tán)
- Throughput: *cực cao* (triệu msg/giây)
- Lưu trữ: *bền vững*, có thể replay
- Consumer: tự quản lý offset
- Phù hợp: data pipeline, event sourcing, analytics

RabbitMQ

- Mô hình: *message broker* truyền thống
- Throughput: *cao* (chục nghìn msg/giây)
- Lưu trữ: xóa sau khi ACK
- Consumer: broker quản lý trạng thái
- Phù hợp: task queue, RPC, workflow phức tạp

Kafka? RabbitMQ?

Lựa chọn dựa trên **bản chất của dữ liệu** và **yêu cầu về throughput, replay, routing**.

Chọn Kafka khi:

- Cần xử lý **lượng sự kiện rất lớn** (log, clickstream, IoT)
- Cần **replay** lại lịch sử sự kiện hoặc nhiều consumer group đọc cùng data
- Xây dựng **data pipeline** hoặc **event sourcing**

Chọn RabbitMQ khi:

- Cần **routing phức tạp** (direct, topic, headers exchange)
- Cần **priority queue** hoặc **dead letter queue** linh hoạt
- Tác vụ cần xác nhận (**acknowledgement**) nghiêm ngặt và không cần replay

Tóm tắt

Message queue là nền tảng không thể thiếu của hệ thống phân tán hiện đại — từ decoupling đến scaling đến fault tolerance.

- ☕ **Producer-Consumer**: phân phối tác vụ, mỗi thông điệp xử lý *đúng một lần*
- ☕ **Publish-Subscribe**: phát sóng sự kiện, mỗi subscriber nhận *bản sao độc lập*
- ☕ **Backpressure**: tín hiệu quan trọng — phải có chiến lược xử lý khi queue đầy
- ☕ **Kafka**: throughput cao, replay được, phù hợp *data streaming & event sourcing*
- ☕ **RabbitMQ**: routing linh hoạt, phù hợp *task queue & workflow phức tạp*

Q & A
